Smart Routing with Learning-based QoS-aware Meta-strategies

Ying Zhang
Markus Fromherz
Lukas Kuhn (*Ludwig Maximilian University*)

October 2004

parc
Palo Alto Research Center

Outline

Message-initiated
Constraint-based Routing

Learning-based
Meta-routing Strategies

Performance Evaluations

Message-initiated Constraint-Based Routing

Routing in Traditional Networks

***Address*-based**
***Table*-driven**
***Statically* configured**

***Off-line* optimization**

MCBR:

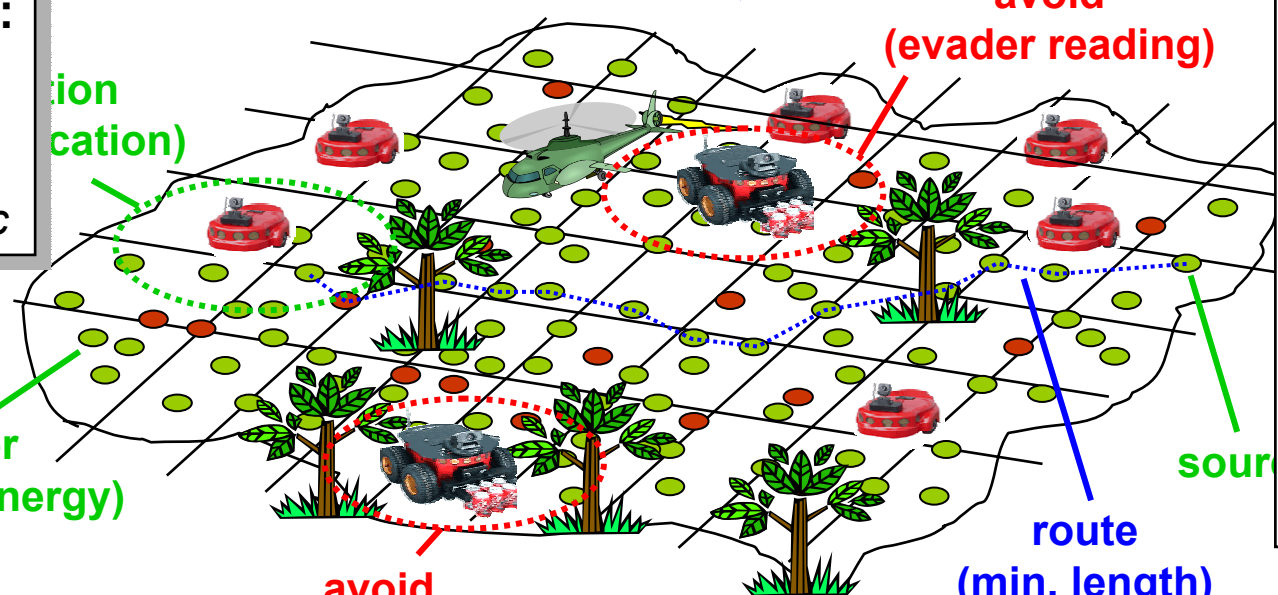
Destination Constraints
Local Route Constraints
Global Route Objectives

Routing in Wireless Ad-hoc Sensor Networks

***Attribute*-based**
***Message*-initiated**
***Dynamically* configured**

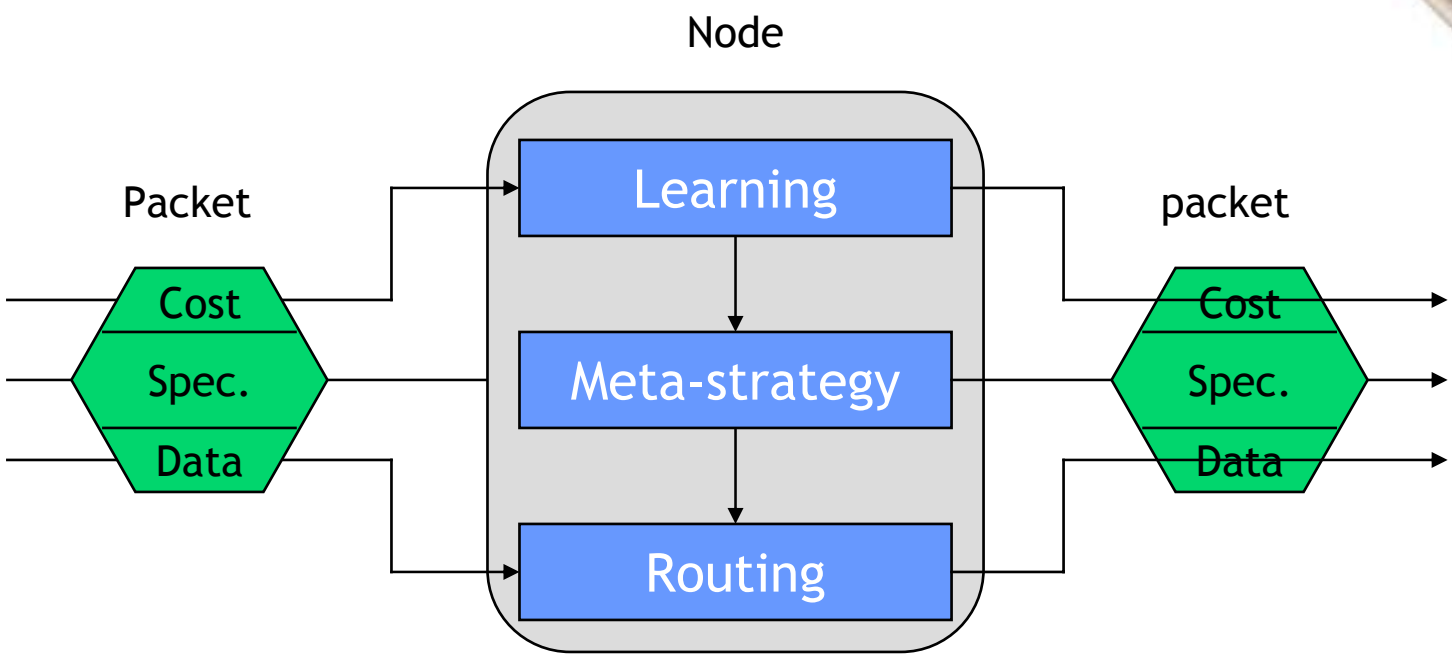
***On-line* optimization**

Properties:
- *Embedded*
- *Dynamic*
- *Dense*
- *Asymmetric*



Attributes:
- *Constants*:
  - Unit Cost
  - Address
  - Group ID
- *Variables*:
  - Location
  - Time
  - Sensors
  - Hops
  - Energy

Learning-based Meta-Routing Strategies



Specification:
- Destination constraints
- Local route constraints
- Global route objectives

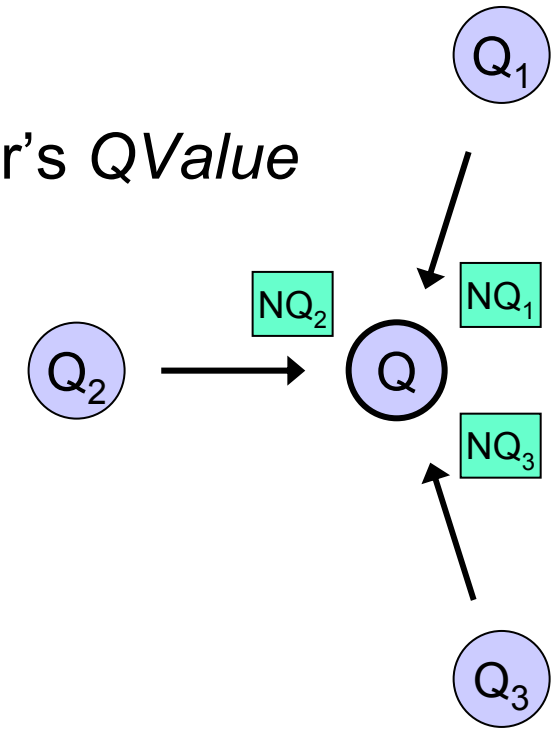
MCBR strategies:
- Real-time Search
- Constrained Flooding
- Adaptive Tree
- Ant-routing

Learning:
- *Q-learning*
- Ant learning

Cost:
- projection of global objective to local node

Q-learning-based Routing

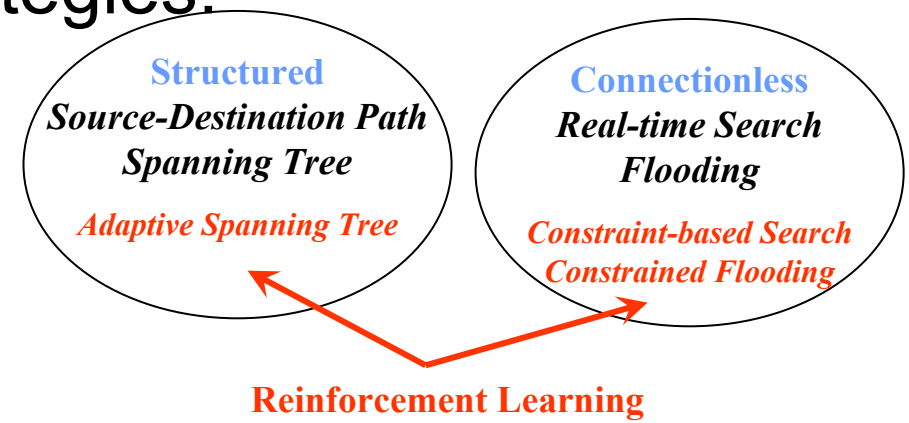
- Routing core: reinforcement learning
  - *QValue*:
    - » estimated cost from the current node to the destination (any cost function)
  - *NQValue*: estimate for each neighbor's *QValue*
    - » or recorded value whenever a message is received from that neighbor
  - learning process:
    - » $QValue = (1-\alpha)*QValue + \alpha*(\min NQValues + lcost)$ (1)
    - » α: learning rate
    - » *lcost*: attribute-based local cost function
  - every message includes the node's *QValue*



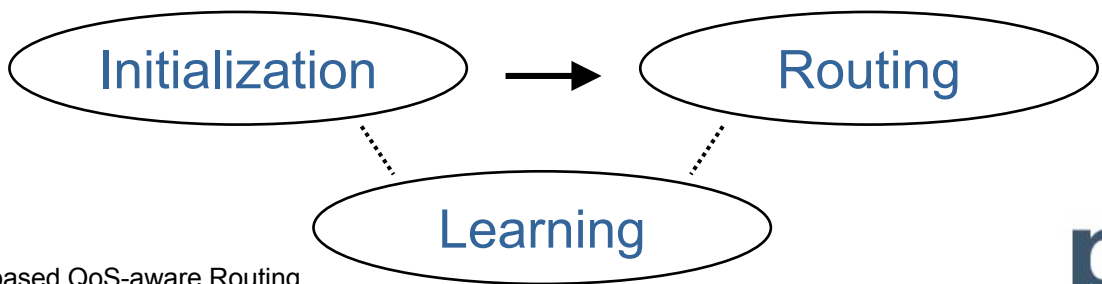
parc
Palo Alto Research Center

Q-learning Meta-strategies

- Routing meta-strategies:
  - search-based
  - flood-based
  - tree-based



- Routing phases
  - Initialization: establishing structures
  - Routing: passing packets
  - Learning: updating *QValue* and *NQValues*



parc
Palo Alto Research Center

Search-based Meta-strategy

- Initialization (required):
  - establish neighborhood structures by sending out ‘Hello’
- Routing:
  - pass received designated packet to neighbor with best *QValue*
- Learning:
  - update $NQValue_i$ for all received or overheard packets from *i*
  - set $NQValue_i$ to max(*NQValues*)+1 if didn’t hear neighbor *i* deliver the designated packet
  - calculate own *QValue* according to (1)

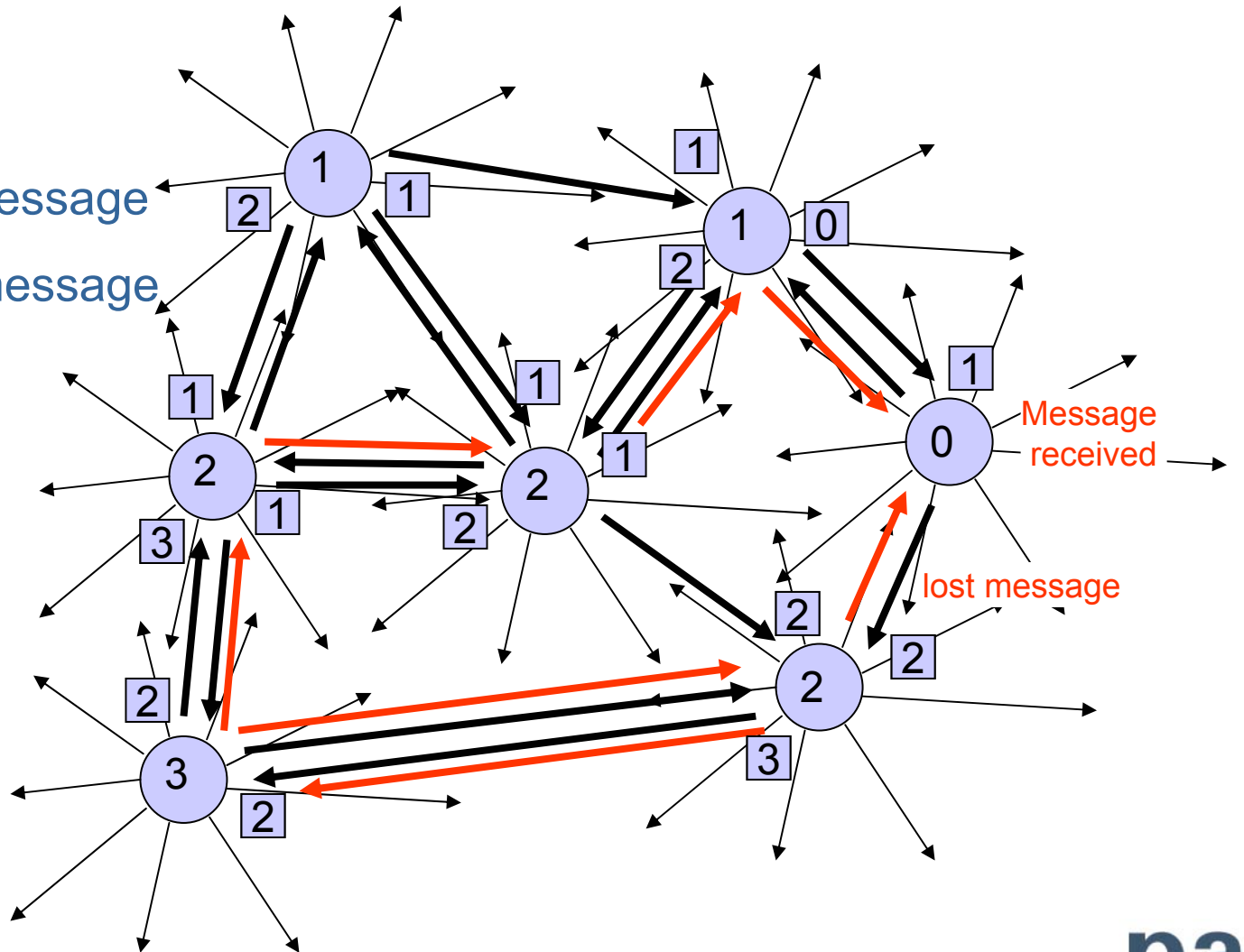
Search Algorithm

Initialization

Learning

Routing 1st message

Routing 2nd message



parc
Palo Alto Research Center

Flood-based Meta-strategy

- Initialization (not required):
  - establish initial *QValues*
  - propagate from destination
- Routing:
  - Δ = data.*QValue* – *QValue* (for received data)
  - if $\Delta + T > 0$ ("temperature" T)
    transmit after $f(\Delta)$ (f decreasing with increasing Δ)
    else ignore
- Learning:
  - update $NQValue_i$ for all received or overheard packets from i
  - calculate own *QValue* according to (1)

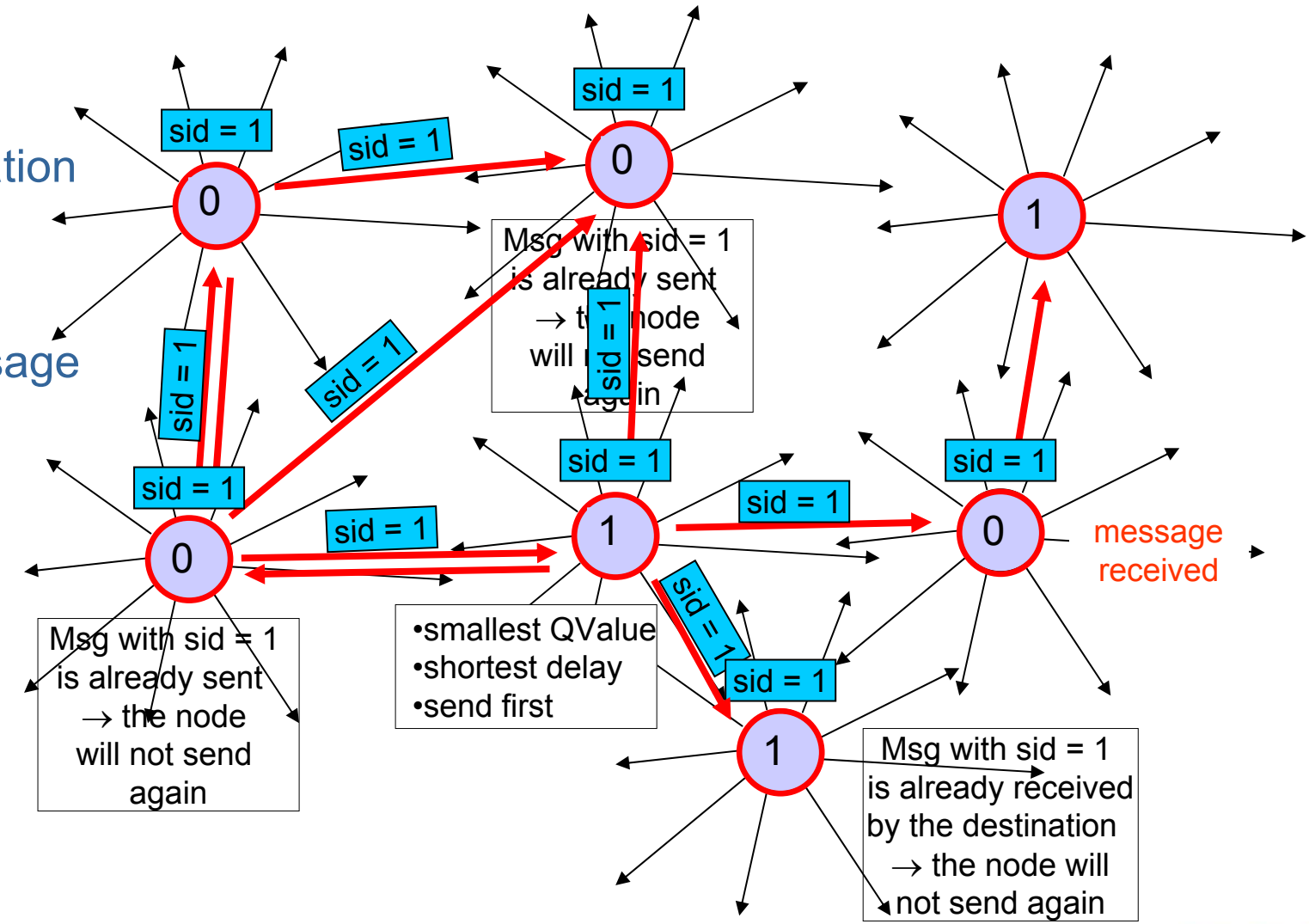
Flood Algorithm

Learning

Initialization

Routing

1st message

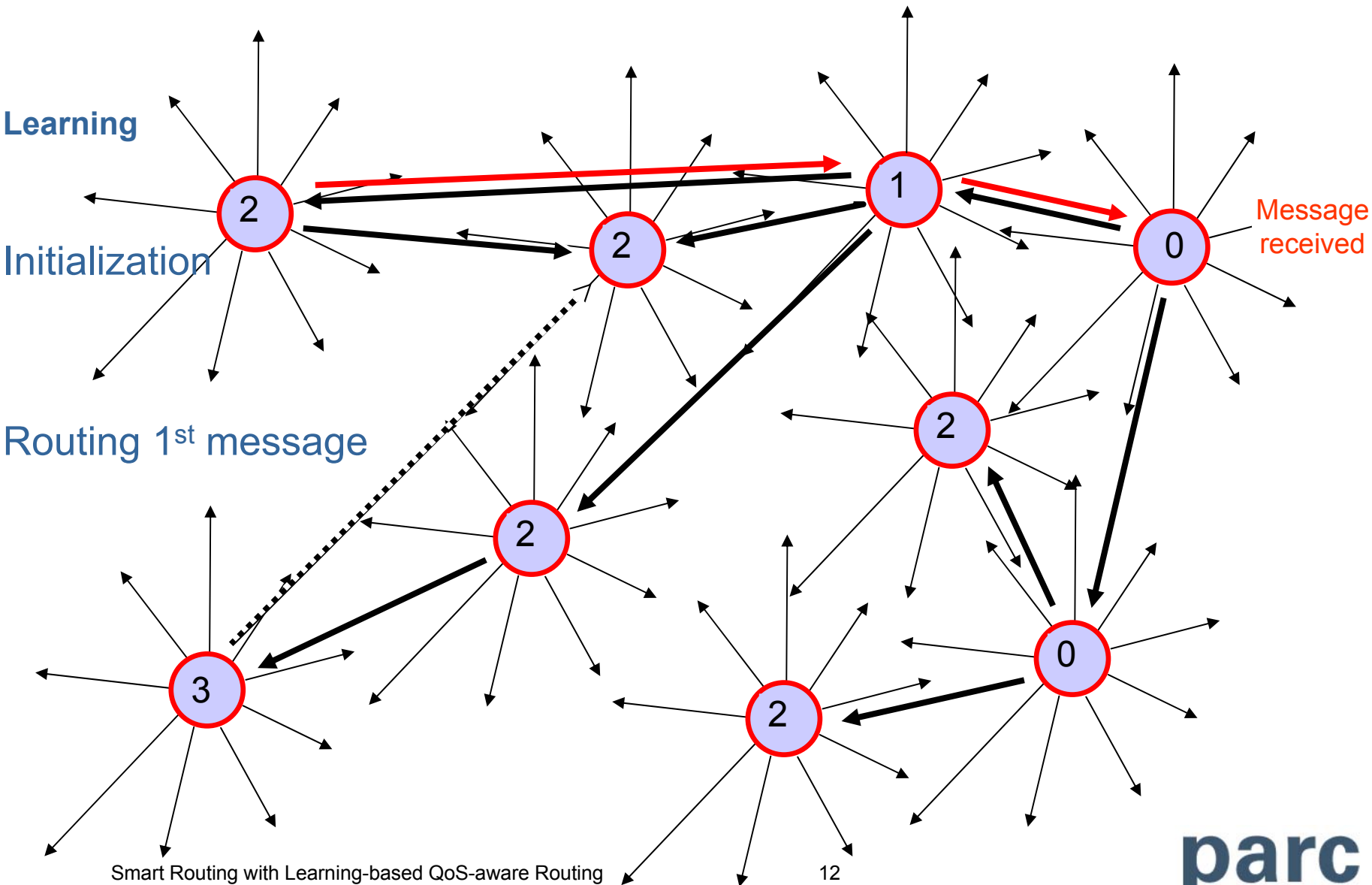


parc Palo Alto Research Center

Tree-based Meta-strategy

- Initialization (required):
  - establish a spanning tree from destination by flooding from destination
  - select node *i* with minimum $NQValue_i$ as parent
- Routing:
  - pass received designated packet on to parent
- Learning:
  - update $NQValue_i$ for all received or overheard packets from *i*
  - set $NQValue_i$ to max(*NQValues*)+1 if didn't hear neighbor *i*
    deliver the designated packet
  - calculate own *QValue* according to (1)
  - reselect node *i* with min $NQValue_i$ as parent

Tree Algorithm



parc
Palo Alto Research Center

MCBR – Take-away Messages

- Flexible framework:
  - generic encoding of application requirements and metrics
  - multiple learning methods possible
  - multiple meta-strategies, re-use of representation & learning core
- Adaptive routing:
  - learning of best routes
  - constantly improving
  - adapting to dynamic network and application
- Low overhead:
  - learning while routing
  - implicit information exchange
  - no extra maintenance needed

Performance Metrics

- ⬇ Latency (s): $T_{received} - T_{sent}$
- ⬆ Throughput (p/s): R/T
- ⬇ Loss Rate: $L/(L+R)$
- ⬆ Success Rate: $\Sigma R/\Sigma S$
- ⬇ Energy Use: ΣU
- ⬆ Energy Efficiency: $\Sigma R/\Sigma U$
- ⬆ Lifetime Predication: $E_{max} - (\underline{U}+\sigma)$

$$\underline{U} \leftarrow \Sigma U/N$$

$$\sigma^2 \leftarrow \Sigma(U-\underline{U})^2/N$$

T: time
R: received packets
L: lost packets
S: original packets
U: used energy
N: total nodes
E: energy

Application Scenario: Pursuer/Evader Game (PEG)

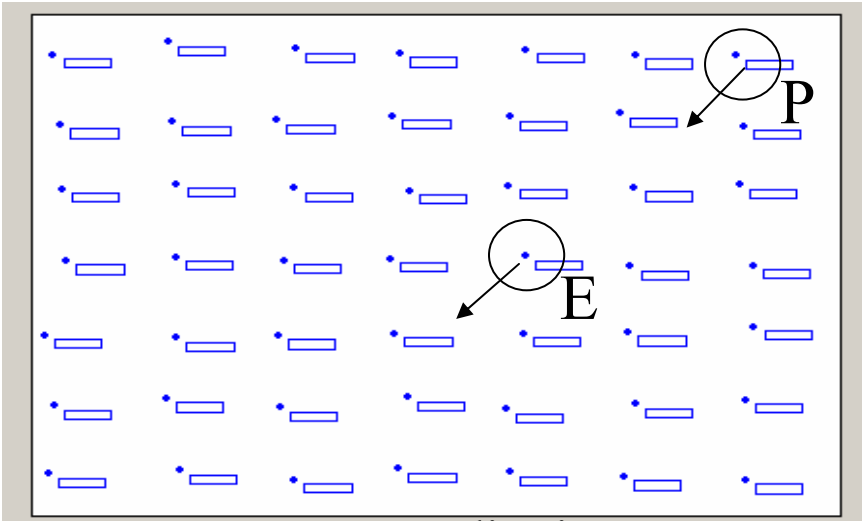
Source:

- Dynamic (0.2d/s)
- Rate: 1p/s

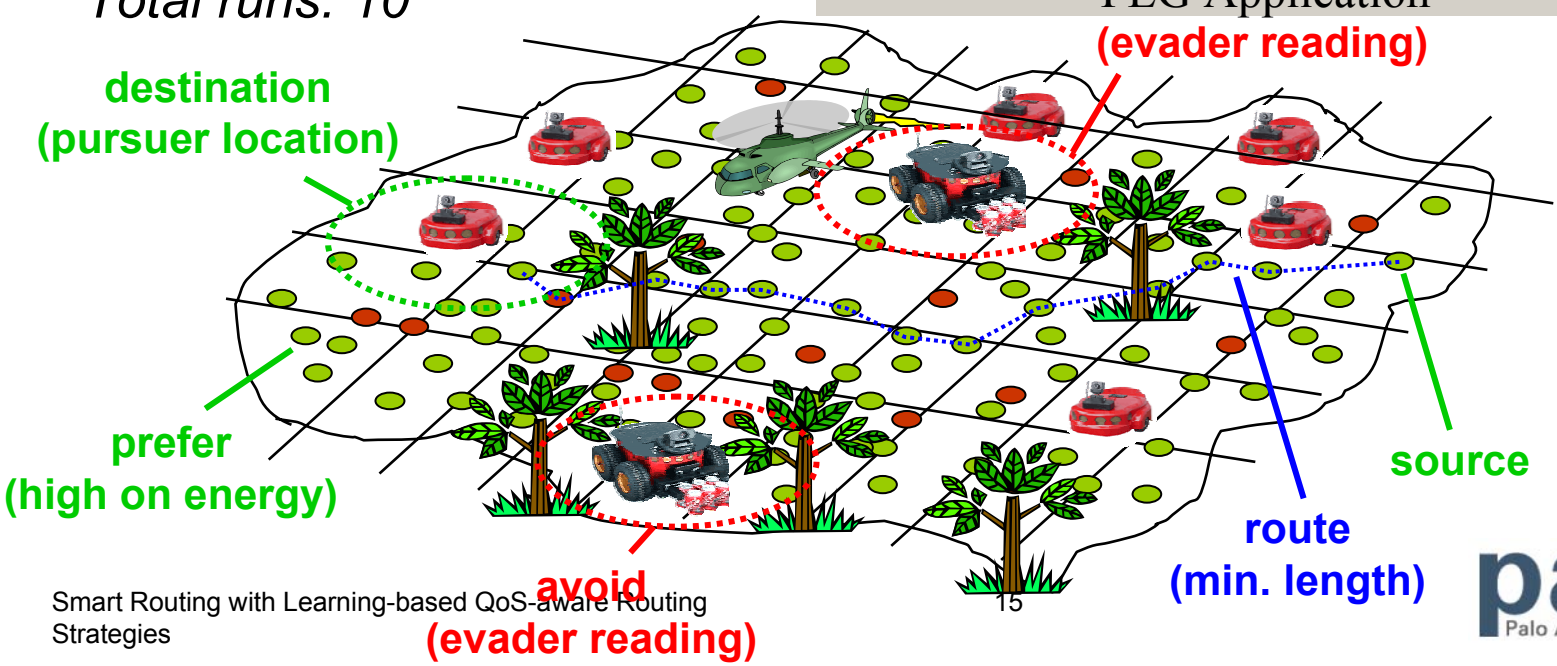
Destination:

- Mobile (0.2d/s)
- Simulation time: 15s

Total runs: 10



PEG Application



parc
Palo Alto Research Center

Simulation Model

Assumptions

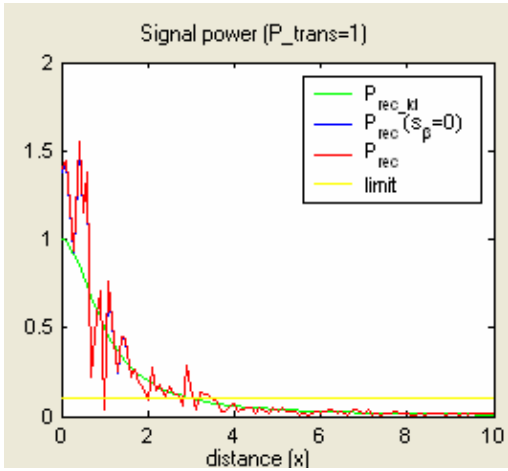
$$P_{rec,ideal}(d) \leftarrow P_{transmit} \frac{1}{1+d^{\gamma}}$$

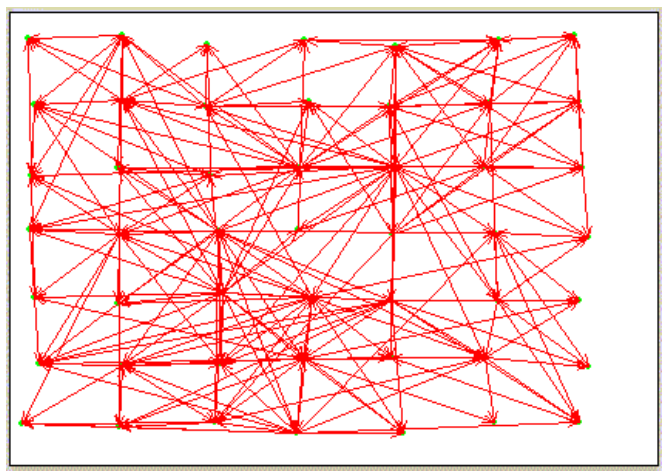
$$P_{rec}(i,j) \leftarrow P_{rec,ideal}(d_{i,j})(1+\alpha(i,j))(1+\beta(t))$$

$$\alpha : N(0,\sigma_{\alpha}), \sigma_{\alpha} \leftarrow 0.45$$

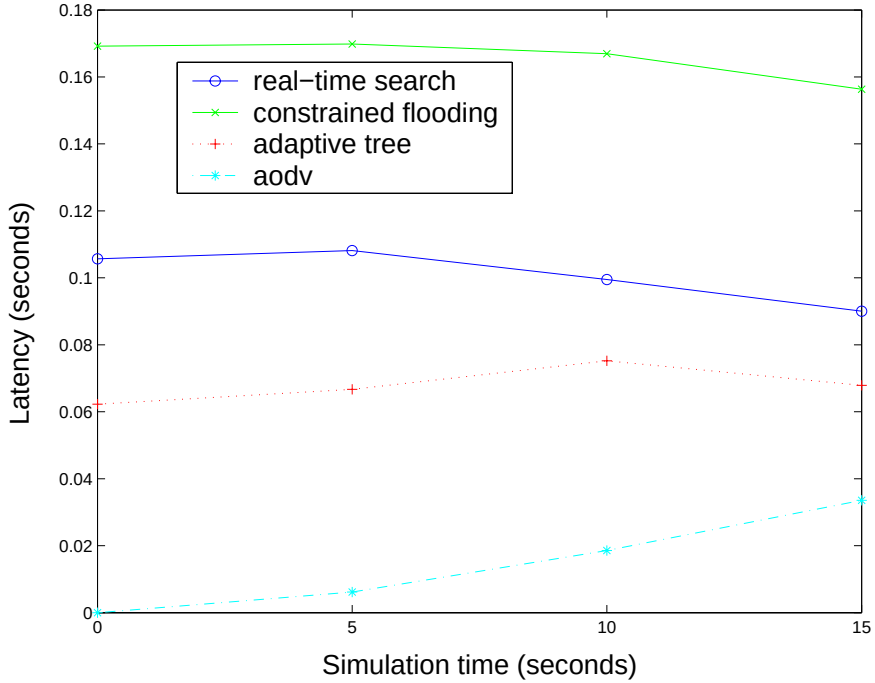
$$\beta : N(0,\sigma_{\beta}), \sigma_{\beta} \leftarrow 0.02$$

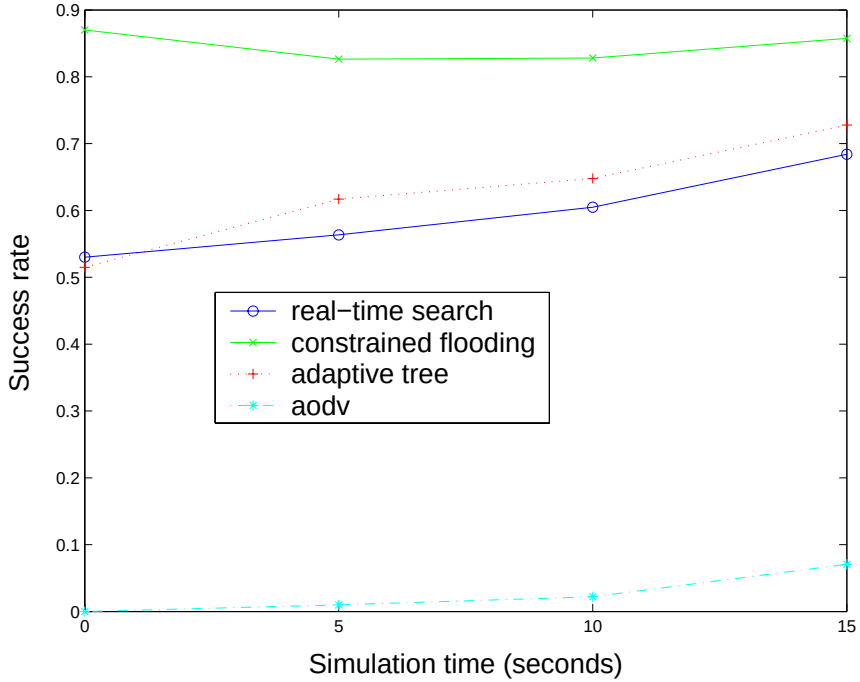
$$i \leftarrow j \Leftrightarrow P_{rec}(i,j) > \Delta$$



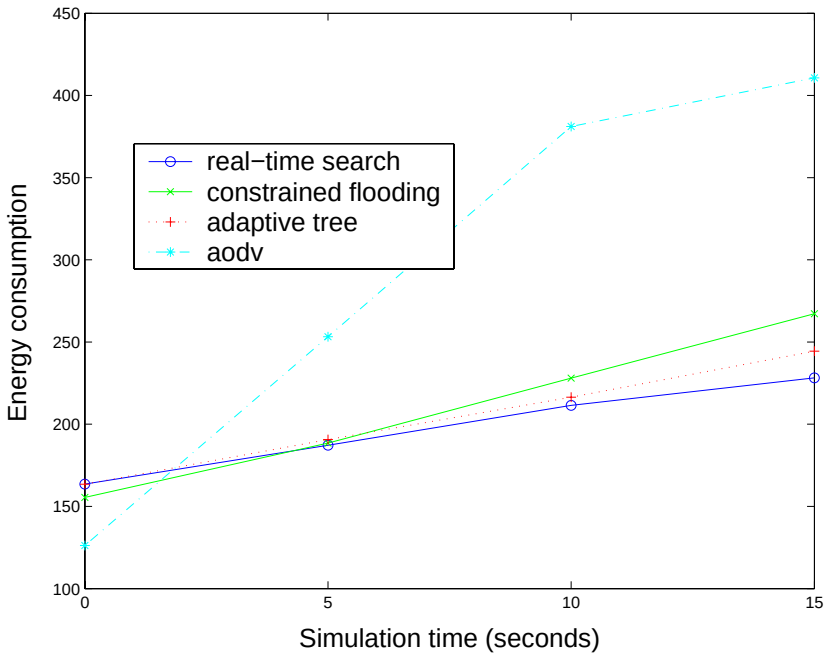


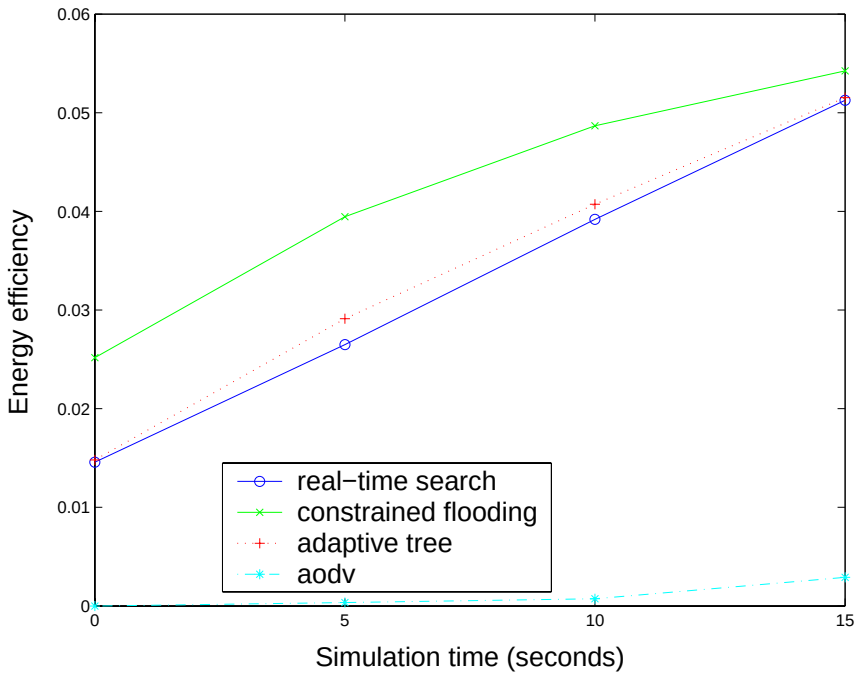
parc Palo Alto Research Center





Performance Comparison on PEG scenario





Conclusions

Routing Spec.
- Geographical
- Energy-aware
- Congestion-aware
- …

Meta-strategies

Q-Learning
- Real-time search
- Constrained flooding
- Adaptive tree

Separate Specification From Routing Strategies

Routing while Learning

Performance Tradeoffs

parc
Palo Alto Research Center